

Ejercicio 1 — Separar los blobs y volver a elegir (k)

Enlaces del notebook en colab

https://colab.research.google.com/drive/15dmQGJnLVBZjTbq_MsQl23OMR9ml5Wkp#scrollTo=8FLZLxLhP4N1

Configuración de los blobs

Primero ejecuté la notebook original en Google Colab y tomé los resultados como baseline. Después modifiqué únicamente la posición de los cinco centros y mantuve los mismos valores de blob_std. La intención fue aumentar la distancia entre grupos sin reducir artificialmente su dispersión.

```
# ORIGINALES
blob_centers = np.array(
    # [[ 0.2, 2.3],
    # [-1.5, 2.3],
    # [-2.8, 1.8],
    # [-2.8, 2.8],
    # [-2.8, 1.3]])
blob_std = np.array([0.4, 0.3, 0.1, 0.1, 0.1])
```

```
# MODIFICADOS

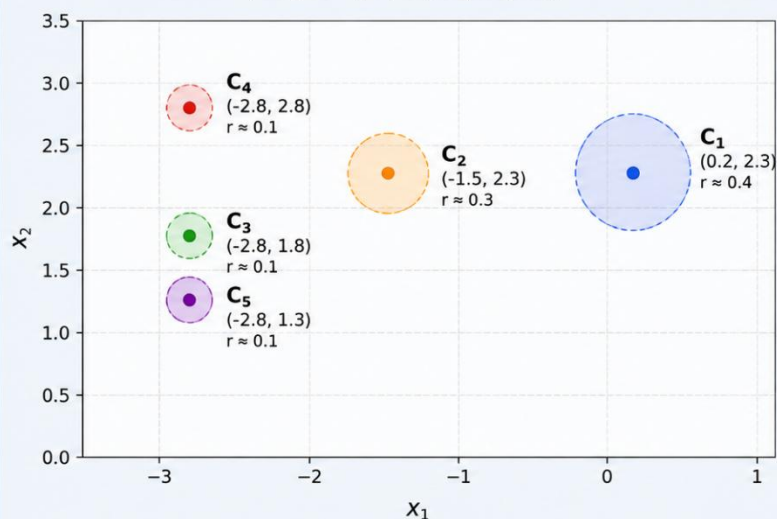
blob_centers = np.array(
    [[ 0.0, 2.5],
    [-1.5, 2.5],
    [-3.0, 3.0],
    [-3.0, 1.8],
    [-3.0, 0.6]])

blob_std = np.array([0.4, 0.3, 0.1, 0.1, 0.1])
```

Dibujo de los centros

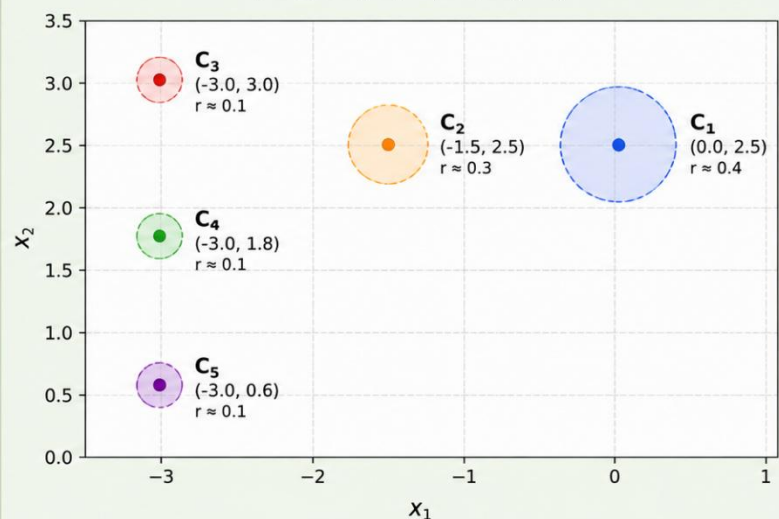
Centros originales

(blob_std = [0.4, 0.3, 0.1, 0.1, 0.1])



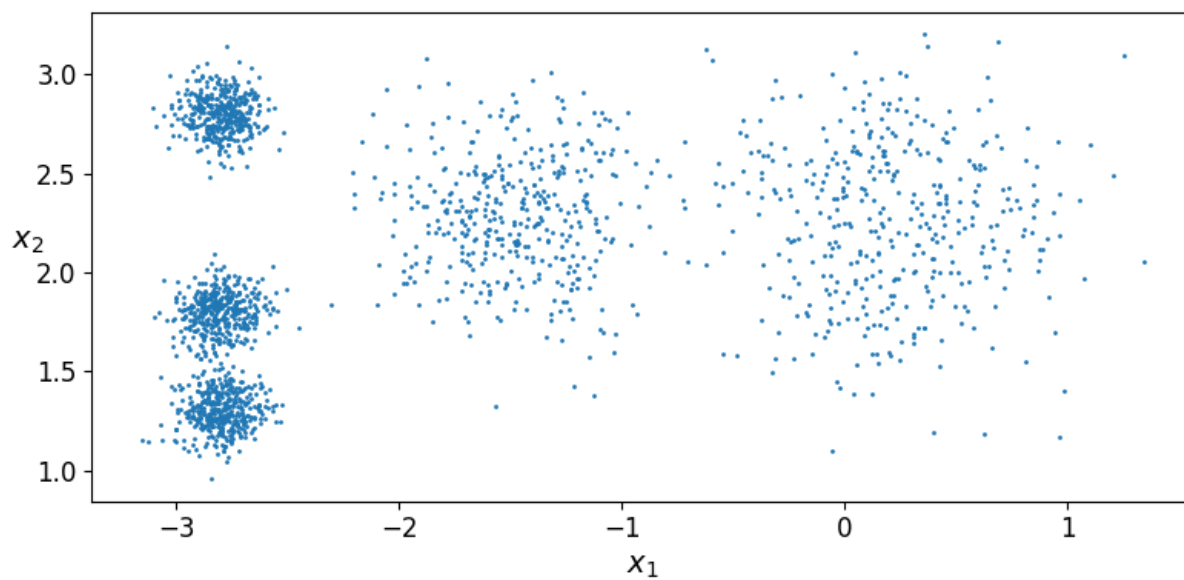
Centros modificados

(blob_std = [0.4, 0.3, 0.1, 0.1, 0.1])

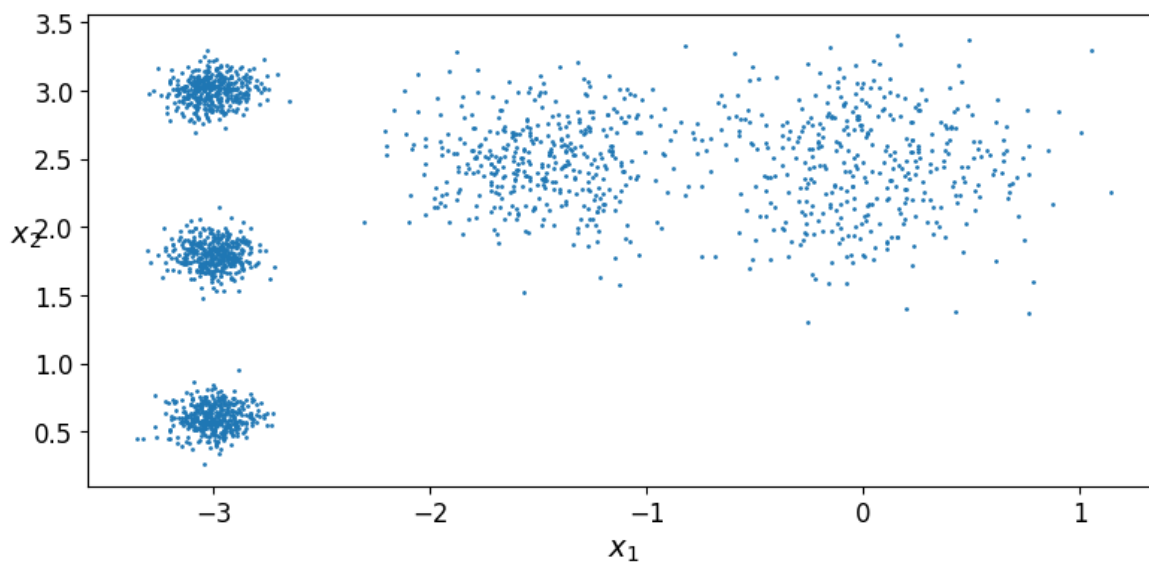


Comparaciones de gráficos

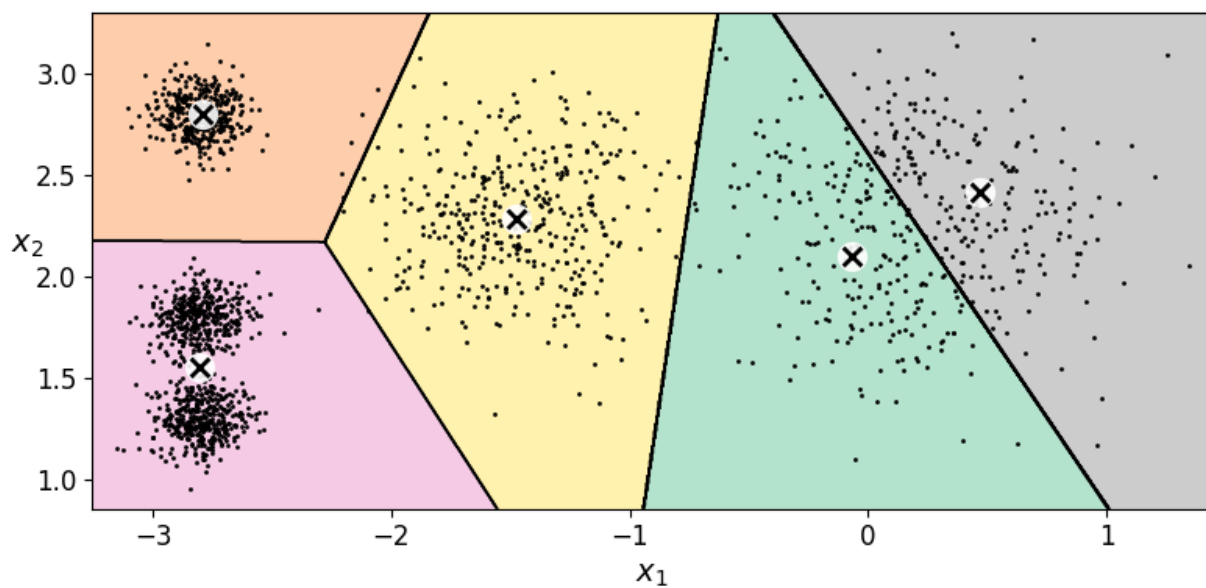
- **Scatter original**



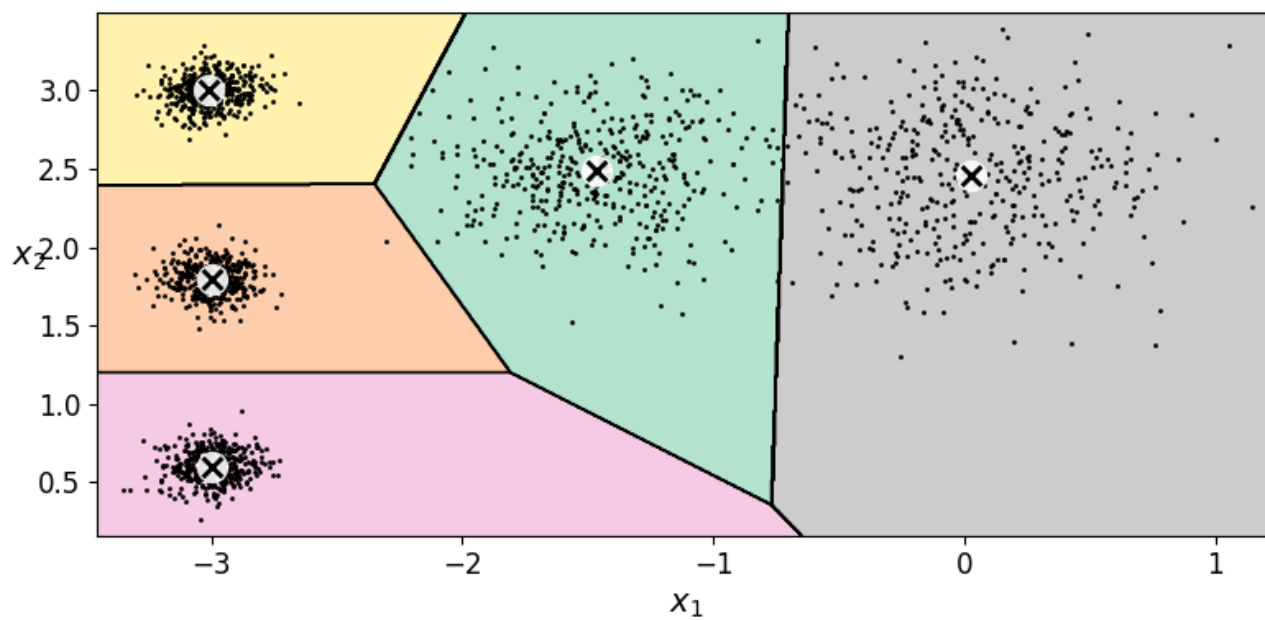
- **Scatter modificado**



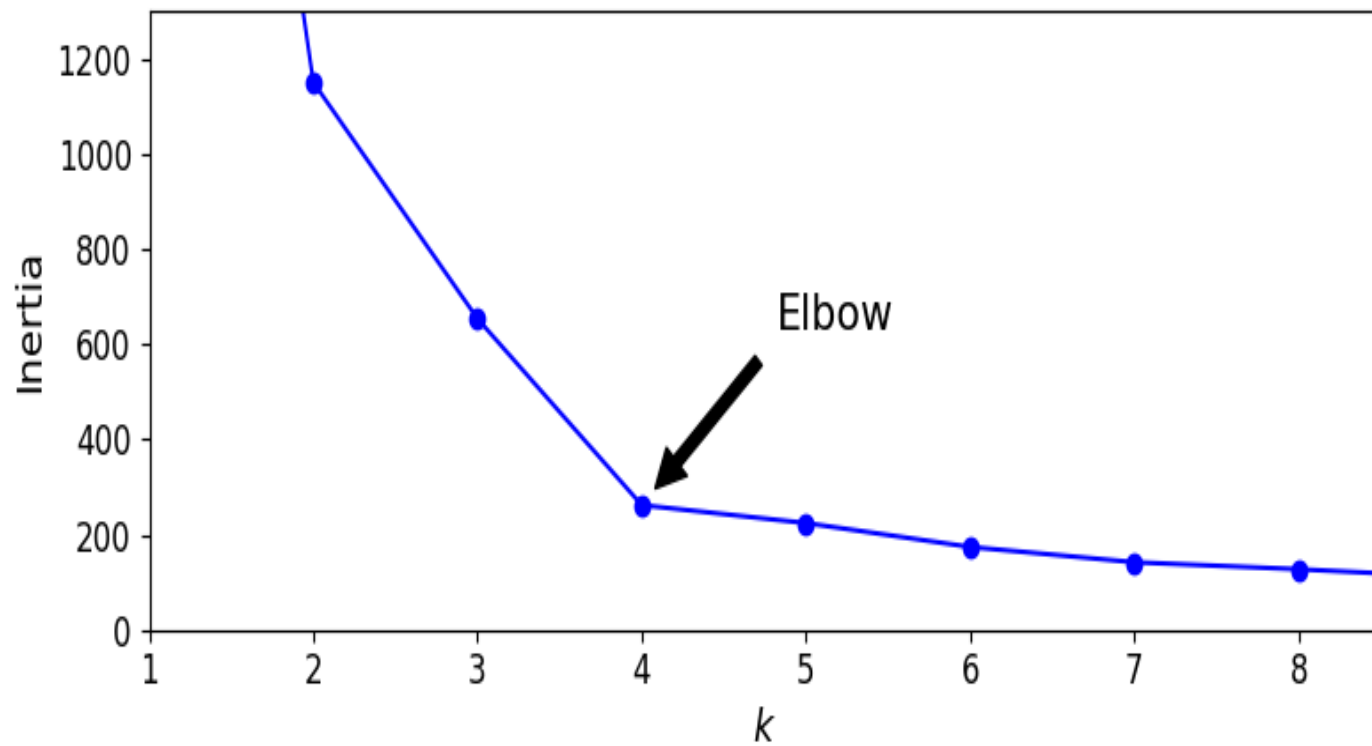
- **Voronoi original**



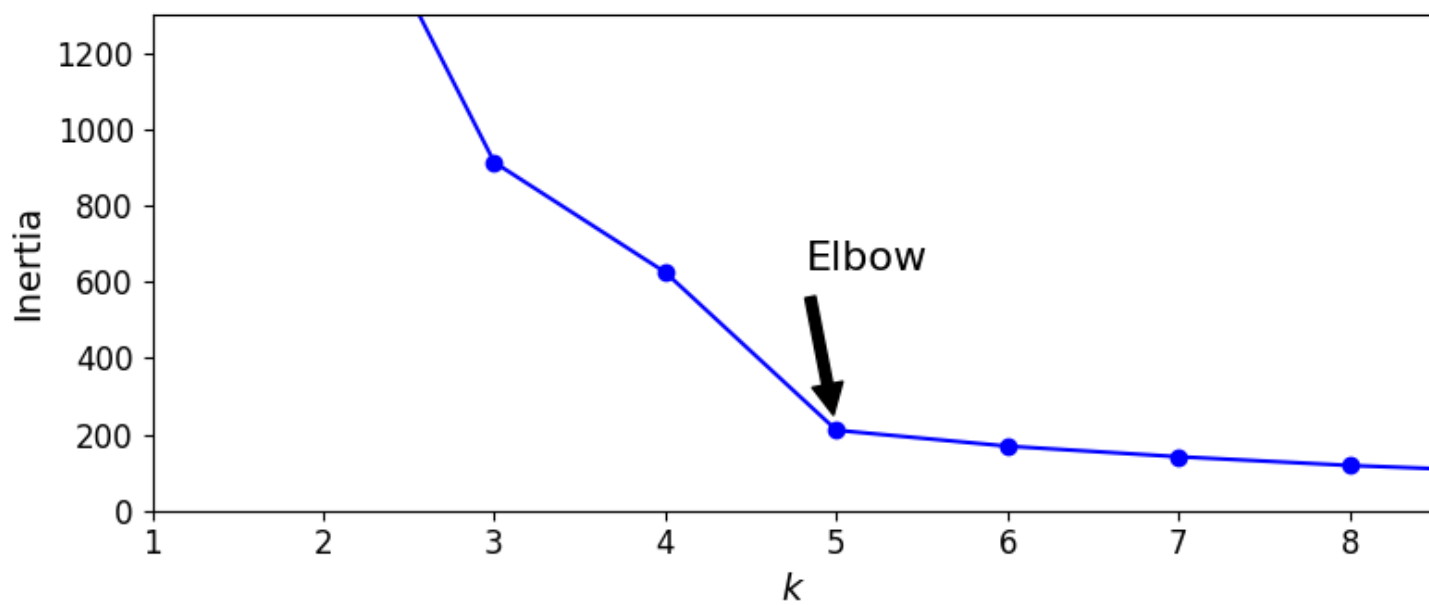
- **Voronoi modificado**



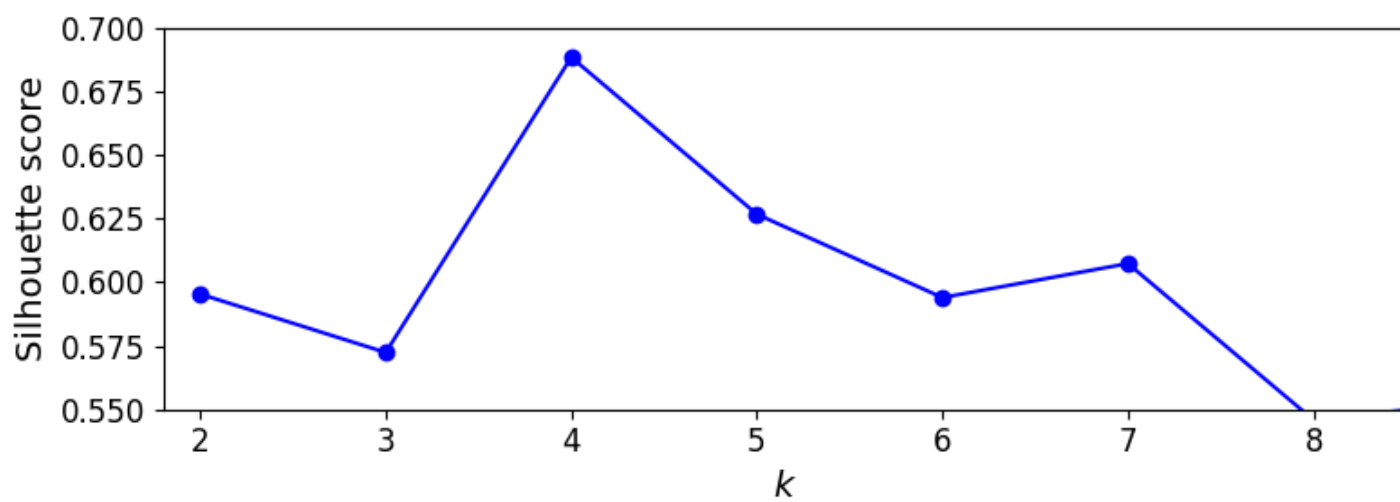
- **Codo original**



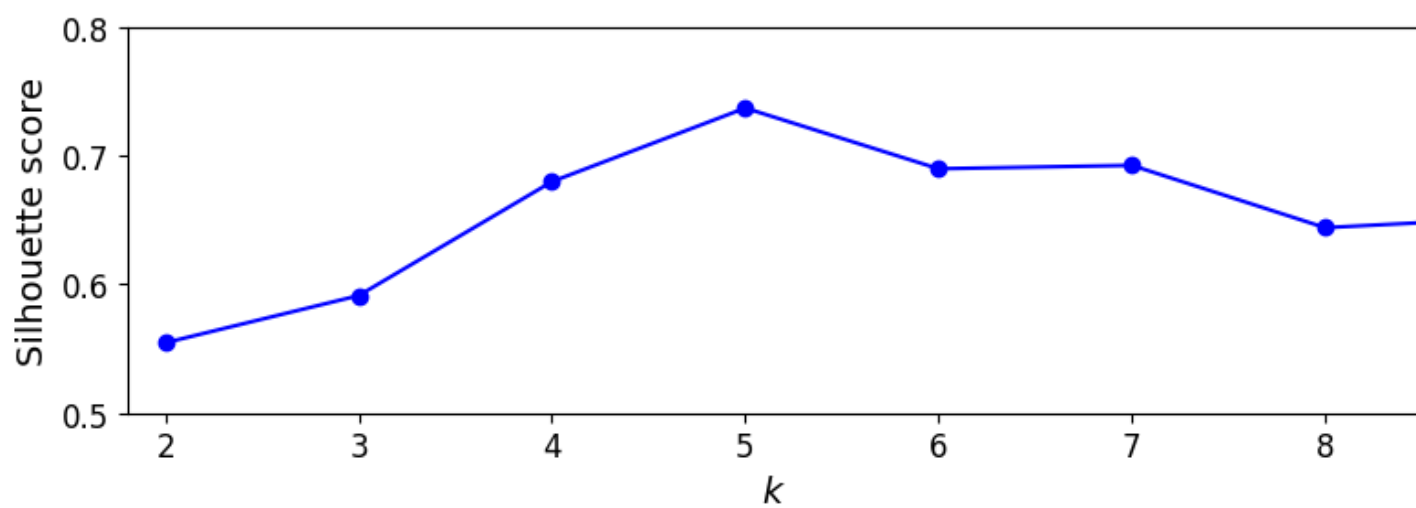
- **Codo modificado**



- **Silueta original**



- **Silueta modificado**



Resultados numéricos

Para comparar los experimentos utilicé las inercias de $k = 3$, 5 y 8, el punto que visualmente marca el codo y el máximo del silhouette score.

Métrica	Original	Modificado
Inercia $k = 3$	653.2167	912.7271
Inercia $k = 5$	224.0743	210.4770
Inercia $k = 8$	127.1314	118.3200
Codo observado	$k = 4$	$k = 5$
Mejor k por Silhouette	$k = 4$	$k = 5$
Silhouette máximo	0.6885	0.7372

El cambio más importante es que el codo pasó de $k = 4$ a $k = 5$ y el máximo de Silhouette también pasó de $k = 4$ a $k = 5$. Además, el Silhouette máximo aumentó de 0.6885 a 0.7372. La inercia con $k = 3$ aumentó porque, al separar más los grupos, representar cinco nubes con solamente tres centroides se vuelve más costoso.

Reporte

En los datos originales de Géron, aunque `make_blobs` genera cinco centros, el método del codo prefiere $k = 4$. Viendo el scatter tiene sentido, porque algunos de los blobs de la izquierda están demasiado cerca entre sí. Para K-means, separar esos grupos como clusters distintos no produce una mejora tan grande en la inercia, así que después de $k = 4$ la curva comienza a aplanarse. El Silhouette confirma algo parecido: su máximo fue 0.6885 en $k = 4$, aunque $k = 5$ también daba una separación razonable.

Para la segunda prueba moví los centros para separar mejor las cinco nubes, pero dejé exactamente los mismos `blob_std`. Con este cambio, el resultado fue más claro: el codo se desplazó a $k = 5$ y el máximo de Silhouette también quedó en $k = 5$, con un score de 0.7372. Además, la inercia para $k = 5$ bajó de 224.0743 a 210.4770. Esto indica que la nueva distribución sí representa mejor cinco grupos distintos y no solamente que visualmente se ve mejor.

Si después de mover los centros el codo todavía hubiera quedado en 4, lo siguiente sería aumentar todavía más la distancia entre los centros o disminuir el `blob_std`. En general, si la distancia entre centros es pequeña comparada con la dispersión de cada nube, los grupos se empiezan a solapar y K-means puede interpretar dos blobs como un solo cluster. En este caso no fue necesario modificar los `std`, porque con separar los centros fue suficiente para que tanto el codo como Silhouette coincidieran en $k = 5$.

Evidencias

01 K-medias.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda

Comandos Código Texto Ejecutar todo

#blob_std = np.array([0.4, 0.3, 0.1, 0.1, 0.1])

MODIFICADOS

blob_centers = np.array([
[0.0, 2.5],
[-1.5, 2.5],
[-3.0, 3.0],
[-3.0, 1.0],
[-3.0, 0.6]
)

blob_std = np.array([0.4, 0.3, 0.1, 0.1, 0.1])

X, y = make_blobs(n_samples=2000, centers=blob_centers,
cluster_std=blob_std, random_state=7)

Now let's plot them:

def plot_clusters(X, y=None):
plt.scatter(X[:, 0], X[:, 1], c=y, s=1)
plt.xlabel("\$x_1\$", fontsize=14)
plt.ylabel("\$x_2\$", fontsize=14, rotation=0)

plt.figure(figsize=(8, 4))
plot_clusters(X)
save_fig("blobs_plot")
plt.show()

Saving figure blobs_plot

Recursos

No tienes una suscripción. [Más información](#)

En este momento no tienes unidades de procesamiento disponibles. Los recursos que se ofrecen sin costo no están garantizados. Compra más unidades [aquí](#).

En tu nivel de uso actual, este tiempo de ejecución puede durar hasta 63 horas 10 minutos.

[Administrar sesiones](#)

¿Quieres acceder a GPU premium? [Actualizar a Google AI Ultra](#)

del backend de Google Compute Engine in Python 3

Se muestran los recursos desde el 12 de septiembre hasta el 12:41 a.m.

RAM de sistema 1.6 / 12.7 GB

Disco 20.3 / 107.7 GB

Cambiar tipo de entorno de ejecución

Variables Terminal 11:54 p.m. Python 3

01 K-medias.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda

Comandos Código Texto Ejecutar todo

%timeit -n 50 KMeans(algorithm="elkan", random_state=42).fit(X)

13.6 ms ± 1.63 ms per loop (mean ± std. dev. of 7 runs, 50 loops each)

%timeit -n 50 KMeans(algorithm="lloyd", random_state=42).fit(X)

8.74 ms ± 1.75 ms per loop (mean ± std. dev. of 7 runs, 50 loops each)

There's no big difference in this case, as the dataset is fairly small.

Finding the optimal number of clusters

What if the number of clusters was set to a lower or greater value than 5?

kmeans_k3 = KMeans(n_clusters=3, random_state=42)
kmeans_k8 = KMeans(n_clusters=8, random_state=42)
plot_clusterer_comparison(kmeans_k3, kmeans_k8, X, "\$k=3\$", "\$k=8\$")
save_fig("bad_n_clusters_plot")
plt.show()

Saving figure bad_n_clusters_plot

Variables Terminal 11:54 p.m. Python 3